

## Lab 2: Attacking Classic Crypto Systems

### Checkpoint 1: Breaking Caesar Cipher

Cipher Text

odroboewscdrolocdcwkbdmyxdbkmdzvkdpybwyeddrobo

#### Approach

- The Caesar cipher is a simple substitution cipher where each letter is shifted by a fixed number of positions in the alphabet
- The encryption and decryption process uses modular arithmetic with modulo 26
- Since there are only 25 possible shifts (excluding shift 0), a brute-force attack is feasible
- The program iterates through all possible shifts (1-25) and displays each potential decryption
- The correct plaintext is identified by looking for meaningful English text

#### Implementation

```
javascript

function ceaserCipher(k, cipher) {
  let text = '';
  for(let i = 0; i < cipher.length; i++) {
    let d = cipher.charCodeAt(i);
    let base = (d >= 65 && d <= 90) ? 65 : 97;
    let char = String.fromCharCode(((d - base - k + 26) % 26) + base);
    text += char;
  }
  return text;
}
```

## Results:

```
PS D:\Downloads\INS-Tasks> node "d:\Downloads\INS-Tasks\Lab2\task1.js"
shift: 1 text: ncqnandvrbcqnknbcbvjaclxwcajlcuyjcoxavxdccqnan
shift: 2 text: mbpmzmcuqabpmjmabauizbkwbzikbtibnwzuwcbpbpmzm
shift: 3 text: laolylbtpzaolilazthyaajvuayhjawshamvytvbaaolyl
shift: 4 text: kznkxkasoyznkhkzyzsgxziutzxgizvrgzluxsuazznkxk
shift: 5 text: jymjwjzrnxymjgjxyxrfwyhtsywfhyuqfyktwrtzyymjwj
shift: 6 text: ixliviymwxlifiwxwqevxgsrxvegxtpexjsvqsyxxlivi
shift: 7 text: hwkhuhxplvwkhehvwpduwfrqwudfwsodwiruprxwwkhuh
shift: 8 text: gvjgtgwokuvjgdguvuoctveqpvtcevrncvhtqtoqvvjgtg
shift: 9 text: fuifsfvnjtuifcftutnbsudpousbduqmbugpsnpvuufsf
shift: 10 text: ethereumisthebestsmartcontractplatformoutthere
shift: 11 text: dsqdqdtlhrsgdadrslzqsbnmsqzbsokzsenqIntssgdqd
shift: 12 text: crfcpcskgrfczccqrqkypramlrpyarnjyrdmpkmsrrfcpc
shift: 13 text: bqebobrjfpqebybpqpjxoqzlkqoxzqmioxqclojlrqebob
shift: 14 text: apdanaqieopdaxaopoiwnpykjpnrwplhwpbknikqppdana
shift: 15 text: zoczmzphdnoczwznonhvmoxjiovmxokgvoajmhjpooczmz
shift: 16 text: ynbylyogcmnbyvymnmgulnwihnlwnjfunzilgionnbyly
shift: 17 text: xmaxkxnfblmaxuxlmlftkmvhgmktvmietmyhkfhnmmaxkx
shift: 18 text: wlzwjwmeaklzwtklkesjlugfljsulhdsxlxjegmllzwjw
shift: 19 text: vkyvivldzjkyvsvjkdriktfekirtkgcrkwfidflkkyviv
shift: 20 text: ujxuhukcyijxuruijicqhjsedjhqsjfbqjvehcekjjxuhu
shift: 21 text: tiwtgtjbxhiwtqthihbpgirdcigprieapiudgbdjiiwtgt
shift: 22 text: shvsfsiawghvpsghgaofhqcbhfoqhdzohtcfacihvsfs
shift: 23 text: rgurerhzvfgurorfgfznegpbagenpgcyngsbezbhggurer
shift: 24 text: qftqdqgyueftqnqefeymdfoazfdmofbxmfradyagfftqdq
shift: 25 text: pespcpfxtdespmpdedxlcenzyeclneawleqzcxzfeespcp
```

Most suitable one is shift-10:

```
shift: 10 text: ethereumisthebestsmartcontractplatformoutthere
```

## Checkpoint 2: Breaking Substitution Cipher

### Cipher Texts

Two different ciphertexts encrypted with different substitution keys.

### Approach

- Each plaintext letter is mapped to a ciphertext letter in a one-to-one relationship
- The mapping is consistent throughout the entire message
- Unlike Caesar cipher, the substitution can be arbitrary rather than just a shift
- English letters have characteristic frequency distributions
- Common letters: e, t, a, o, i, n, s, h, r, d, l, c, u, m, w, f, g, y, p, b, v, k, j, x, q, z
- The program analyzes letter frequencies in the ciphertext and maps them to English letter frequencies

### 3. Implementation Details

```
def frequency_analysis(text):
    freq = {}
    total_letters = 0

    for c in text.lower():
        if c in string.ascii_lowercase:
            freq[c] = freq.get(c, 0) + 1
            total_letters += 1

    freq_percent = {char: (count/total_letters)*100 for char, count in freq.items()}
    sorted_freq = sorted(freq_percent.items(), key=lambda x: x[1], reverse=True)

    return sorted_freq
```

```
def apply_mapping(text, mapping):
    """Apply character mapping to text"""
    result = []
    for c in text:
        if c.lower() in mapping:
            # Preserve case
            if c.isupper():
                result.append(mapping[c.lower()].upper())
            else:
                result.append(mapping[c.lower()])
        else:
            result.append(c)
    return ''.join(result)
```

## Results

### Cipher 2 Decryption:

bilbo was very rich and very peculiar, and had been the wonder of the shire for sixty years, ever since his remarkable disappearance and unexpected return. the riches he had brought back from his travels had now become a local legend, and it was popularly believed, whatever the old folk might say, that the hill at bag end was full of tunnels stuffed with treasure. and if that was not enough for fame, there was also his prolonged vigour to marvel at. time wore on, but it seemed to have little effect on mr. baggins. at ninety he was much the same as at fifty. at ninety-nine they began to call him well-preserved; but unchanged would have been nearer the mark. there were some

that shook their heads and thought this was too much of a good thing; it seemed unfair that anyone should possess (apparently) perpetual youth as well as (reputedly) inexhaustible wealth. it will have to be paid for, they said. it isn't natural, and trouble will come of it! but so far trouble had not come; and as mr. baggins was generous with his money, most people were willing to forgive him his oddities and his good fortune. he

remained on visiting terms with his relatives (except, of course, the sackville-bagginses), and he had many devoted admirers among the hobbits of poor and

unimportant families. but he had no close friends, until some of his younger cousins began to grow up. the eldest of these, and bilbo's favourite, was young frodo baggins. when bilbo was ninety-nine he adopted frodo as his heir, and brought him to live at bag end; and the hopes of the sackville-bagginses were finally dashed. bilbo and frodo happened to have the same birthday, september 22nd. you had better come and live here, frodo my lad, said bilbo one day; and then we can celebrate our birthday-parties comfortably together. at that time frodo was still in his tweens, as the hobbits called the irresponsible twenties between childhood and coming of age at thirty-three

Cipher 1 Decryption:

in a particular and, in each case, different way, these four were indispensable to him--yugo amaryl, because of his quick understanding of the principles of psychohistory and of his imaginative probings into new areas. it was comforting to know that if anything happened to seldon himself before the mathematics of the field could be completely worked out--and how slowly it proceeded, and how mountainous the obstacles--there would at least remain one good mind that would continue the research

### **Comparative Analysis**

Cipher 1 was significantly easier to break for several reasons:

1. Cipher 1 is shorter, making frequency analysis more challenging but the patterns emerge more clearly with manual verification.
2. Cipher 1 contains more common English words and patterns that are easily recognizable after partial decryption.
3. The letter distribution in Cipher 1 more closely matched expected English frequencies, requiring fewer manual corrections.
4. Cipher 1 contained more recognizable word patterns and common phrases that helped verify the decryption.

### **Challenges Faced:**

1. Letters like q, x, z have low frequencies and are challenging to map accurately.
2. The presence of punctuation helped identify word boundaries but sometimes interfered with frequency calculations.
3. Real text doesn't always perfectly match theoretical frequency distributions, requiring

### **Key Vulnerabilities:**

1. Caesar Cipher:
  - Limited keyspace (only 25 possible keys)
  - Preservation of letter frequency patterns
  - Easy brute-force attack
2. Substitution Cipher:
  - Preservation of individual letter frequencies
  - Consistent mapping throughout the message
  - Predictable patterns in English language

This lab successfully demonstrated the weaknesses of classical cryptographic systems:

1. Caesar Cipher is completely insecure against brute-force attacks due to its extremely small keyspace.
2. Substitution Ciphers, while more secure than Caesar, are vulnerable to frequency analysis attacks, especially with sufficient ciphertext length.